

**Proof** Immediate from Lemmas 15.2 and 15.3. ■

### Exercises

#### 15.3-1

Explain why, in the proof of Lemma 15.2, if  $x.\text{freq} = b.\text{freq}$ , then we must have  $a.\text{freq} = b.\text{freq} = x.\text{freq} = y.\text{freq}$ .

#### 15.3-2

Prove that a non-full binary tree cannot correspond to an optimal prefix-free code.

#### 15.3-3

What is an optimal Huffman code for the following set of frequencies, based on the first 8 Fibonacci numbers?

a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21

Can you generalize your answer to find the optimal code when the frequencies are the first  $n$  Fibonacci numbers?

#### 15.3-4

Prove that the total cost  $B(T)$  of a full binary tree  $T$  for a code equals the sum, over all internal nodes, of the combined frequencies of the two children of the node.

#### 15.3-5

Given an optimal prefix-free code on a set  $C$  of  $n$  characters, you wish to transmit the code itself using as few bits as possible. Show how to represent any optimal prefix-free code on  $C$  using only  $2n - 1 + n \lceil \lg n \rceil$  bits. (*Hint*: Use  $2n - 1$  bits to specify the structure of the tree, as discovered by a walk of the tree.)

#### 15.3-6

Generalize Huffman's algorithm to ternary codewords (i.e., codewords using the symbols 0, 1, and 2), and prove that it yields optimal ternary codes.

#### 15.3-7

A data file contains a sequence of 8-bit characters such that all 256 characters are about equally common: the maximum character frequency is less than twice the minimum character frequency. Prove that Huffman coding in this case is no more efficient than using an ordinary 8-bit fixed-length code.

#### 15.3-8

Show that no lossless (invertible) compression scheme can guarantee that for every input file, the corresponding output file is shorter. (*Hint*: Compare the number of possible files with the number of possible encoded files.)

---

## 15.4 Offline caching

Computer systems can decrease the time to access data by storing a subset of the main memory in the *cache*: a small but faster memory. A cache organizes data into *cache blocks* typically comprising 32, 64, or 128 bytes. You can also think of main memory as a cache for disk-resident data in a virtual-memory system. Here, the blocks are called *pages*, and 4096 bytes is a typical size.

As a computer program executes, it makes a sequence of memory requests. Say that there are  $n$  memory requests, to data in blocks  $b_1, b_2, \dots, b_n$ , in that order. The blocks in the access sequence might not be distinct, and indeed, any given block is usually accessed multiple times. For example, a program that accesses four distinct blocks  $p, q, r, s$  might make a sequence of requests to blocks  $s, q, s, q, q, s, p, p, r, s, s, q, p, r, q$ . The cache can hold up to some fixed number  $k$  of cache blocks. It starts out empty before the first request. Each request causes at most one block to enter the cache and at most one block to be evicted from the cache. Upon a request for block  $b_i$ , any one of three scenarios may occur:

1. Block  $b_i$  is already in the cache, due to a previous request for the same block. The cache remains unchanged. This situation is known as a *cache hit*.
2. Block  $b_i$  is not in the cache at that time, but the cache contains fewer than  $k$  blocks. In this case, block  $b_i$  is placed into the cache, so that the cache contains one more block than it did before the request.
3. Block  $b_i$  is not in the cache at that time and the cache is full: it contains  $k$  blocks. Block  $b_i$  is placed into the cache, but before that happens, some other block in the cache must be evicted from the cache in order to make room.

The latter two situations, in which the requested block is not already in the cache, are called *cache misses*. The goal is to minimize the number of cache misses or, equivalently, to maximize the number of cache hits, over the entire sequence of  $n$  requests. A cache miss that occurs while the cache holds fewer than  $k$  blocks—that is, as the cache is first being filled up—is known as a *compulsory miss*, since no prior decision could have kept the requested block in the cache. When a cache miss occurs and the cache is full, ideally the choice of which block to evict should allow for the smallest possible number of cache misses over the entire sequence of future requests.

Typically, caching is an online problem. That is, the computer has to decide which blocks to keep in the cache without knowing the future requests. Here, however, let's consider the offline version of this problem, in which the computer knows in advance the entire sequence of  $n$  requests and the cache size  $k$ , with a goal of minimizing the total number of cache misses.

To solve this offline problem, you can use a greedy strategy called *furthest-in-future*, which chooses to evict the block in the cache whose next access in the request sequence comes furthest in the future. Intuitively, this strategy makes sense: if you're not going to need something for a while, why keep it around? We'll show that the furthest-in-future strategy is indeed optimal by showing that the offline caching problem exhibits optimal substructure and that furthest-in-future has the greedy-choice property.

Now, you might be thinking that since the computer usually doesn't know the sequence of requests in advance, there is no point in studying the offline problem. Actually, there is. In some situations, you do know the sequence of requests in advance. For example, if you view the main memory as the cache and the full set of data as residing on disk (or a solid-state drive), there are algorithms that plan out the entire set of reads and writes in advance. Furthermore, we can use the number of cache misses produced by an optimal algorithm as a baseline for comparing how well online algorithms perform. We'll do just that in Section 27.3.

Offline caching can even model real-world problems. For example, consider a scenario where you know in advance a fixed schedule of  $n$  events at known locations. Events may occur at a location multiple times, not necessarily consecutively. You are managing a group of  $k$  agents, you need to ensure that you have one agent at each location when an event occurs, and you want to minimize the number of times that agents have to move. Here, the agents are like the blocks, the events are like the requests, and moving an agent is akin to a cache miss.

### Optimal substructure of offline caching

To show that the offline problem exhibits optimal substructure, let's define the subproblem  $(C, i)$  as processing requests for blocks  $b_i, b_{i+1}, \dots, b_n$  with cache configuration  $C$  at the time that the request for block  $b_i$  occurs, that is,  $C$  is a subset of the set of blocks such that  $|C| \leq k$ . A solution to subproblem  $(C, i)$  is a sequence of decisions that specifies which block to evict (if any) upon each request for blocks  $b_i, b_{i+1}, \dots, b_n$ . An optimal solution to subproblem  $(C, i)$  minimizes the number of cache misses.

Consider an optimal solution  $S$  to subproblem  $(C, i)$ , and let  $C'$  be the contents of the cache after processing the request for block  $b_i$  in solution  $S$ . Let  $S'$  be the subsolution of  $S$  for the resulting subproblem  $(C', i + 1)$ . If the request for  $b_i$  results in a cache hit, then the cache remains unchanged, so that  $C' = C$ . If the request for block  $b_i$  results in a cache miss, then the contents of the cache change, so that  $C' \neq C$ . We claim that in either case,  $S'$  is an optimal solution to subproblem  $(C', i + 1)$ . Why? If  $S'$  is not an optimal solution to subproblem  $(C', i + 1)$ , then there exists another solution  $S''$  to subproblem  $(C', i + 1)$  that makes fewer cache misses than  $S'$ . Combining  $S''$  with the decision of  $S$  at the request for

block  $b_i$  yields another solution that makes fewer cache misses than  $S$ , which contradicts the assumption that  $S$  is an optimal solution to subproblem  $(C, i)$ .

To quantify a recursive solution, we need a little more notation. Let  $R_{C,i}$  be the set of all cache configurations that can immediately follow configuration  $C$  after processing a request for block  $b_i$ . If the request results in a cache hit, then the cache remains unchanged, so that  $R_{C,i} = \{C\}$ . If the request for  $b_i$  results in a cache miss, then there are two possibilities. If the cache is not full ( $|C| < k$ ), then the cache is filling up and the only choice is to insert  $b_i$  into the cache, so that  $R_{C,i} = \{C \cup \{b_i\}\}$ . If the cache is full ( $|C| = k$ ) upon a cache miss, then  $R_{C,i}$  contains  $k$  potential configurations: one for each candidate block in  $C$  that could be evicted and replaced by block  $b_i$ . In this case,  $R_{C,i} = \{(C - \{x\}) \cup \{b_i\} : x \in C\}$ . For example, if  $C = \{p, q, r\}$ ,  $k = 3$ , and block  $s$  is requested, then  $R_{C,i} = \{\{p, q, s\}, \{p, r, s\}, \{q, r, s\}\}$ .

Let  $\text{miss}(C, i)$  denote the minimum number of cache misses in a solution for subproblem  $(C, i)$ . Here is a recurrence for  $\text{miss}(C, i)$ :

$$\text{miss}(C, i) = \begin{cases} 0 & \text{if } i = n \text{ and } b_n \in C, \\ 1 & \text{if } i = n \text{ and } b_n \notin C, \\ \text{miss}(C, i + 1) & \text{if } i < n \text{ and } b_i \in C, \\ 1 + \min \{\text{miss}(C', i + 1) : C' \in R_{C,i}\} & \text{if } i < n \text{ and } b_i \notin C. \end{cases}$$

### Greedy-choice property

To prove that the furthest-in-future strategy yields an optimal solution, we need to show that optimal offline caching exhibits the greedy-choice property. Combined with the optimal-substructure property, the greedy-choice property will prove that furthest-in-future produces the minimum possible number of cache misses.

#### **Theorem 15.5 (Optimal offline caching has the greedy-choice property)**

Consider a subproblem  $(C, i)$  when the cache  $C$  contains  $k$  blocks, so that it is full, and a cache miss occurs. When block  $b_i$  is requested, let  $z = b_m$  be the block in  $C$  whose next access is furthest in the future. (If some block in the cache will never again be referenced, then consider any such block to be block  $z$ , and add a dummy request for block  $z = b_m = b_{n+1}$ .) Then evicting block  $z$  upon a request for block  $b_i$  is included in some optimal solution for the subproblem  $(C, i)$ .

**Proof** Let  $S$  be an optimal solution to subproblem  $(C, i)$ . If  $S$  evicts block  $z$  upon the request for block  $b_i$ , then we are done, since we have shown that some optimal solution includes evicting  $z$ .

So now suppose that optimal solution  $S$  evicts some other block  $x$  when block  $b_i$  is requested. We'll construct another solution  $S'$  to subproblem  $(C, i)$  which, upon

the request for  $b_i$ , evicts block  $z$  instead of  $x$  and induces no more cache misses than  $S$  does, so that  $S'$  is also optimal. Because different solutions may yield different cache configurations, denote by  $C_{S,j}$  the configuration of the cache under solution  $S$  just before the request for some block  $b_j$ , and likewise for solution  $S'$  and  $C_{S',j}$ . We'll show how to construct  $S'$  with the following properties:

1. For  $j = i + 1, \dots, m$ , let  $D_j = C_{S,j} \cap C_{S',j}$ . Then,  $|D_j| \geq k - 1$ , so that the cache configurations  $C_{S,j}$  and  $C_{S',j}$  differ by at most one block. If they differ, then  $C_{S,j} = D_j \cup \{z\}$  and  $C_{S',j} = D_j \cup \{y\}$  for some block  $y \neq z$ .
2. For each request of blocks  $b_i, \dots, b_{m-1}$ , if solution  $S$  has a cache hit, then solution  $S'$  also has a cache hit.
3. For all  $j > m$ , the cache configurations  $C_{S,j}$  and  $C_{S',j}$  are identical.
4. Over the sequence of requests for blocks  $b_i, \dots, b_m$ , the number of cache misses produced by solution  $S'$  is at most the number of cache misses produced by solution  $S$ .

We'll prove inductively that these properties hold for each request.

1. We proceed by induction on  $j$ , for  $j = i + 1, \dots, m$ . For the base case, the initial caches  $C_{S,i}$  and  $C_{S',i}$  are identical. Upon the request for block  $b_i$ , solution  $S$  evicts  $x$  and solution  $S'$  evicts  $z$ . Thus, cache configurations  $C_{S,i+1}$  and  $C_{S',i+1}$  differ by just one block,  $C_{S,i+1} = D_{i+1} \cup \{z\}$ ,  $C_{S',i+1} = D_{i+1} \cup \{x\}$ , and  $x \neq z$ .

The inductive step defines how solution  $S'$  behaves upon a request for block  $b_j$  for  $i + 1 \leq j \leq m - 1$ . The inductive hypothesis is that property 1 holds when  $b_j$  is requested. Because  $z = b_m$  is the block in  $C_{S,i}$  whose next reference is furthest in the future, we know that  $b_j \neq z$ . We consider several scenarios:

- If  $C_{S,j} = C_{S',j}$  (so that  $|D_j| = k$ ), then solution  $S'$  makes the same decision upon the request for  $b_j$  as  $S$  makes, so that  $C_{S,j+1} = C_{S',j+1}$ .
- If  $|D_j| = k - 1$  and  $b_j \in D_j$ , then both caches already contain block  $b_j$ , and both solutions  $S$  and  $S'$  have cache hits. Therefore,  $C_{S,j+1} = C_{S,j}$  and  $C_{S',j+1} = C_{S',j}$ .
- If  $|D_j| = k - 1$  and  $b_j \notin D_j$ , then because  $C_{S,j} = D_j \cup \{z\}$  and  $b_j \neq z$ , solution  $S$  has a cache miss. It evicts either block  $z$  or some block  $w \in D_j$ .
  - If solution  $S$  evicts block  $z$ , then  $C_{S,j+1} = D_j \cup \{b_j\}$ . There are two cases, depending on whether  $b_j = y$ :
    - If  $b_j = y$ , then solution  $S'$  has a cache hit, so that  $C_{S',j+1} = C_{S',j} = D_j \cup \{b_j\}$ . Thus,  $C_{S,j+1} = C_{S',j+1}$ .
    - If  $b_j \neq y$ , then solution  $S'$  has a cache miss. It evicts block  $y$ , so that  $C_{S',j+1} = D_j \cup \{b_j\}$ , and again  $C_{S,j+1} = C_{S',j+1}$ .

- If solution  $S$  evicts some block  $w \in D_j$ , then  $C_{S,j+1} = (D_j - \{w\}) \cup \{b_j, z\}$ . Once again, there are two cases, depending on whether  $b_j = y$ :
  - If  $b_j = y$ , then solution  $S'$  has a cache hit, so that  $C_{S',j+1} = C_{S',j} = D_j \cup \{b_j\}$ . Since  $w \in D_j$  and  $w$  was not evicted by solution  $S'$ , we have  $w \in C_{S',j+1}$ . Therefore,  $w \notin D_{j+1}$  and  $b_j \in D_{j+1}$ , so that  $D_{j+1} = (D_j - \{w\}) \cup \{b_j\}$ . Thus,  $C_{S,j+1} = D_{j+1} \cup \{z\}$ ,  $C_{S',j+1} = D_{j+1} \cup \{w\}$ , and because  $w \neq z$ , property 1 holds when block  $b_{j+1}$  is requested. (In other words, block  $w$  replaces block  $y$  in property 1.)
  - If  $b_j \neq y$ , then solution  $S'$  has a cache miss. It evicts block  $w$ , so that  $C_{S',j+1} = (D_j - \{w\}) \cup \{b_j, y\}$ . Therefore, we have that  $D_{j+1} = (D_j - \{w\}) \cup \{b_j\}$  and so  $C_{S,j+1} = D_{j+1} \cup \{z\}$  and  $C_{S',j+1} = D_{j+1} \cup \{y\}$ .
- 2. In the above discussion about maintaining property 1, solution  $S$  may have a cache hit in only the first two cases, and solution  $S'$  has a cache hit in these cases if and only if  $S$  does.
- 3. If  $C_{S,m} = C_{S',m}$ , then solution  $S'$  makes the same decision upon the request for block  $z = b_m$  as  $S$  makes, so that  $C_{S,m+1} = C_{S',m+1}$ . If  $C_{S,m} \neq C_{S',m}$ , then by property 1,  $C_{S,m} = D_m \cup \{z\}$  and  $C_{S',m} = D_m \cup \{y\}$ , where  $y \neq z$ . In this case, solution  $S$  has a cache hit, so that  $C_{S,m+1} = C_{S,m} = D_m \cup \{z\}$ . Solution  $S'$  evicts block  $y$  and brings in block  $z$ , so that  $C_{S',m+1} = D_m \cup \{z\} = C_{S,m+1}$ . Thus, regardless of whether or not  $C_{S,m} = C_{S',m}$ , we have  $C_{S,m+1} = C_{S',m+1}$ , and starting with the request for block  $b_{m+1}$ , solution  $S'$  simply makes the same decisions as  $S$ .
- 4. By property 2, upon the requests for blocks  $b_i, \dots, b_{m-1}$ , whenever solution  $S$  has a cache hit, so does  $S'$ . Only the request for block  $b_m = z$  remains to be considered. If  $S$  has a cache miss upon the request for  $b_m$ , then regardless of whether  $S'$  has a cache hit or a cache miss, we are done:  $S'$  has at most the same number of cache misses as  $S$ .

So now suppose that  $S$  has a cache hit and  $S'$  has a cache miss upon the request for  $b_m$ . We'll show that there exists a request for at least one of blocks  $b_{i+1}, \dots, b_{m-1}$  in which the request results in a cache miss for  $S$  and a cache hit for  $S'$ , thereby compensating for what happens upon the request for block  $b_m$ . The proof is by contradiction. Assume that no request for blocks  $b_{i+1}, \dots, b_{m-1}$  results in a cache miss for  $S$  and a cache hit for  $S'$ .

We start by observing that once the caches  $C_{S,j}$  and  $C_{S',j}$  are equal for some  $j > i$ , they remain equal thereafter. Observe also that if  $b_m \in C_{S,m}$  and  $b_m \notin C_{S',m}$ , then  $C_{S,m} \neq C_{S',m}$ . Therefore, solution  $S$  cannot have evicted block  $z$  upon the requests for blocks  $b_i, \dots, b_{m-1}$ , for if it had, then these two

cache configurations would be equal. The remaining possibility is that upon each of these requests, we had  $C_{S,j} = D_j \cup \{z\}$ ,  $C_{S',j} = D_j \cup \{y\}$  for some block  $y \neq z$ , and solution  $S$  evicted some block  $w \in D_j$ . Moreover, since none of these requests resulted in a cache miss for  $S$  and a cache hit for  $S'$ , the case of  $b_j = y$  never occurred. That is, for every request of blocks  $b_{i+1}, \dots, b_{m-1}$ , the requested block  $b_j$  was never the block  $y \in C_{S',j} - C_{S,j}$ . In these cases, after processing the request, we had  $C_{S',j+1} = D_{j+1} \cup \{y\}$ : the difference between the two caches did not change. Now, let's go back to the request for block  $b_i$ , where afterward, we had  $C_{S',i+1} = D_{i+1} \cup \{x\}$ . Because every succeeding request until requesting block  $b_m$  did not change the difference between the caches, we had  $C_{S',j} = D_j \cup \{x\}$  for  $j = i + 1, \dots, m$ .

By definition, block  $z = b_m$  is requested after block  $x$ . That means at least one of blocks  $b_{i+1}, \dots, b_{m-1}$  is block  $x$ . But for  $j = i + 1, \dots, m$ , we have  $x \in C_{S',j}$  and  $x \notin C_{S,j}$ , so that at least one of these requests had a cache hit for  $S'$  and a cache miss for  $S$ , a contradiction. We conclude that if solution  $S$  has a cache hit and solution  $S'$  has a cache miss upon the request for block  $b_m$ , then some earlier request had the opposite result, and so solution  $S'$  produces no more cache misses than solution  $S$ . Since  $S$  is assumed to be optimal,  $S'$  is optimal as well. ■

Along with the optimal-substructure property, Theorem 15.5 tells us that the furthest-in-future strategy yields the minimum number of cache misses.

## Exercises

### 15.4-1

Write pseudocode for a cache manager that uses the furthest-in-future strategy. It should take as input a set  $C$  of blocks in the cache, the number of blocks  $k$  that the cache can hold, a sequence  $b_1, b_2, \dots, b_n$  of requested blocks, and the index  $i$  into the sequence for the block  $b_i$  being requested. For each request, it should print out whether a cache hit or cache miss occurs, and for each cache miss, it should also print out which block, if any, is evicted.

### 15.4-2

Real cache managers do not know the future requests, and so they often use the past to decide which block to evict. The *least-recently-used*, or *LRU*, strategy evicts the block that, of all blocks currently in the cache, was the least recently requested. (You can think of LRU as “furthest-in-past.”) Give an example of a request sequence in which the LRU strategy is not optimal, by showing that it induces more cache misses than the furthest-in-future strategy does on the same request sequence.



**15.4-3**

Professor Croesus suggests that in the proof of Theorem 15.5, the last clause in property 1 can change to  $C_{S',j} = D_j \cup \{x\}$  or, equivalently, require the block  $y$  given in property 1 to always be the block  $x$  evicted by solution  $S$  upon the request for block  $b_i$ . Show where the proof breaks down with this requirement.

**15.4-4**

This section has assumed that at most one block is placed into the cache whenever a block is requested. You can imagine, however, a strategy in which multiple blocks may enter the cache upon a single request. Show that for every solution that allows multiple blocks to enter the cache upon each request, there is another solution that brings in only one block upon each request and is at least as good.

---

**Problems**
**15-1 Coin changing**

Consider the problem of making change for  $n$  cents using the smallest number of coins. Assume that each coin's value is an integer.

- a. Describe a greedy algorithm to make change consisting of quarters, dimes, nickels, and pennies. Prove that your algorithm yields an optimal solution.
- b. Suppose that the available coins are in denominations that are powers of  $c$ : the denominations are  $c^0, c^1, \dots, c^k$  for some integers  $c > 1$  and  $k \geq 1$ . Show that the greedy algorithm always yields an optimal solution.
- c. Give a set of coin denominations for which the greedy algorithm does not yield an optimal solution. Your set should include a penny so that there is a solution for every value of  $n$ .
- d. Give an  $O(nk)$ -time algorithm that makes change for any set of  $k$  different coin denominations using the smallest number of coins, assuming that one of the coins is a penny.

**15-2 Scheduling to minimize average completion time**

You are given a set  $S = \{a_1, a_2, \dots, a_n\}$  of tasks, where task  $a_i$  requires  $p_i$  units of processing time to complete. Let  $C_i$  be the **completion time** of task  $a_i$ , that is, the time at which task  $a_i$  completes processing. Your goal is to minimize the average completion time, that is, to minimize  $(1/n) \sum_{i=1}^n C_i$ . For example, suppose that there are two tasks  $a_1$  and  $a_2$  with  $p_1 = 3$  and  $p_2 = 5$ , and consider the schedule



in which  $a_2$  runs first, followed by  $a_1$ . Then we have  $C_2 = 5$ ,  $C_1 = 8$ , and the average completion time is  $(5 + 8)/2 = 6.5$ . If task  $a_1$  runs first, however, then we have  $C_1 = 3$ ,  $C_2 = 8$ , and the average completion time is  $(3 + 8)/2 = 5.5$ .

- a.* Give an algorithm that schedules the tasks so as to minimize the average completion time. Each task must run nonpreemptively, that is, once task  $a_i$  starts, it must run continuously for  $p_i$  units of time until it is done. Prove that your algorithm minimizes the average completion time, and analyze the running time of your algorithm.
- b.* Suppose now that the tasks are not all available at once. That is, each task cannot start until its *release time*  $b_i$ . Suppose also that tasks may be *preempted*, so that a task can be suspended and restarted at a later time. For example, a task  $a_i$  with processing time  $p_i = 6$  and release time  $b_i = 1$  might start running at time 1 and be preempted at time 4. It might then resume at time 10 but be preempted at time 11, and it might finally resume at time 13 and complete at time 15. Task  $a_i$  has run for a total of 6 time units, but its running time has been divided into three pieces. Give an algorithm that schedules the tasks so as to minimize the average completion time in this new scenario. Prove that your algorithm minimizes the average completion time, and analyze the running time of your algorithm.

---

## Chapter notes

Much more material on greedy algorithms can be found in Lawler [276] and Papadimitriou and Steiglitz [353]. The greedy algorithm first appeared in the combinatorial optimization literature in a 1971 article by Edmonds [131].

The proof of correctness of the greedy algorithm for the activity-selection problem is based on that of Gavril [179].

Huffman codes were invented in 1952 [233]. Lelewer and Hirschberg [294] surveys data-compression techniques known as of 1987.

The furthest-in-future strategy was proposed by Belady [41], who suggested it for virtual-memory systems. Alternative proofs that furthest-in-future is optimal appear in articles by Lee et al. [284] and Van Roy [443].

Imagine that you join Buff's Gym. Buff charges a membership fee of \$60 per month, plus \$3 for every time you use the gym. Because you are disciplined, you visit Buff's Gym every day during the month of November. On top of the \$60 monthly charge for November, you pay another  $3 \times \$30 = \$90$  that month. Although you can think of your fees as a flat fee of \$60 and another \$90 in daily fees, you can think about it in another way. All together, you pay \$150 over 30 days, or an average of \$5 per day. When you look at your fees in this way, you are *amortizing* the monthly fee over the 30 days of the month, spreading it out at \$2 per day.

You can do the same thing when you analyze running times. In an *amortized analysis*, you average the time required to perform a sequence of data-structure operations over all the operations performed. With amortized analysis, you show that if you average over a sequence of operations, then the average cost of an operation is small, even though a single operation within the sequence might be expensive. Amortized analysis differs from average-case analysis in that probability is not involved. An amortized analysis guarantees the *average performance of each operation in the worst case*.

The first three sections of this chapter cover the three most common techniques used in amortized analysis. Section 16.1 starts with aggregate analysis, in which you determine an upper bound  $T(n)$  on the total cost of a sequence of  $n$  operations. The average cost per operation is then  $T(n)/n$ . You take the average cost as the amortized cost of each operation, so that all operations have the same amortized cost.

Section 16.2 covers the accounting method, in which you determine an amortized cost of each operation. When there is more than one type of operation, each type of operation may have a different amortized cost. The accounting method overcharges some operations early in the sequence, storing the overcharge as “pre-